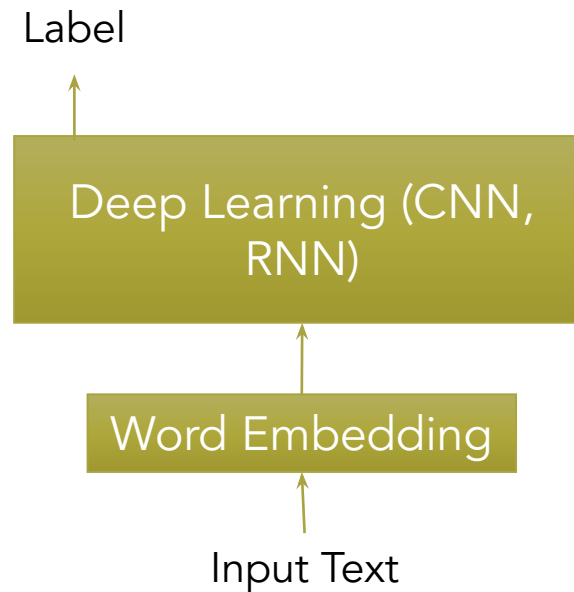


Lab Work 2: Text Classification with Deep Learning

IF 5153 Advanced Natural Language Processing

Sem1 26/27

General Flow of Text Classification (DL)



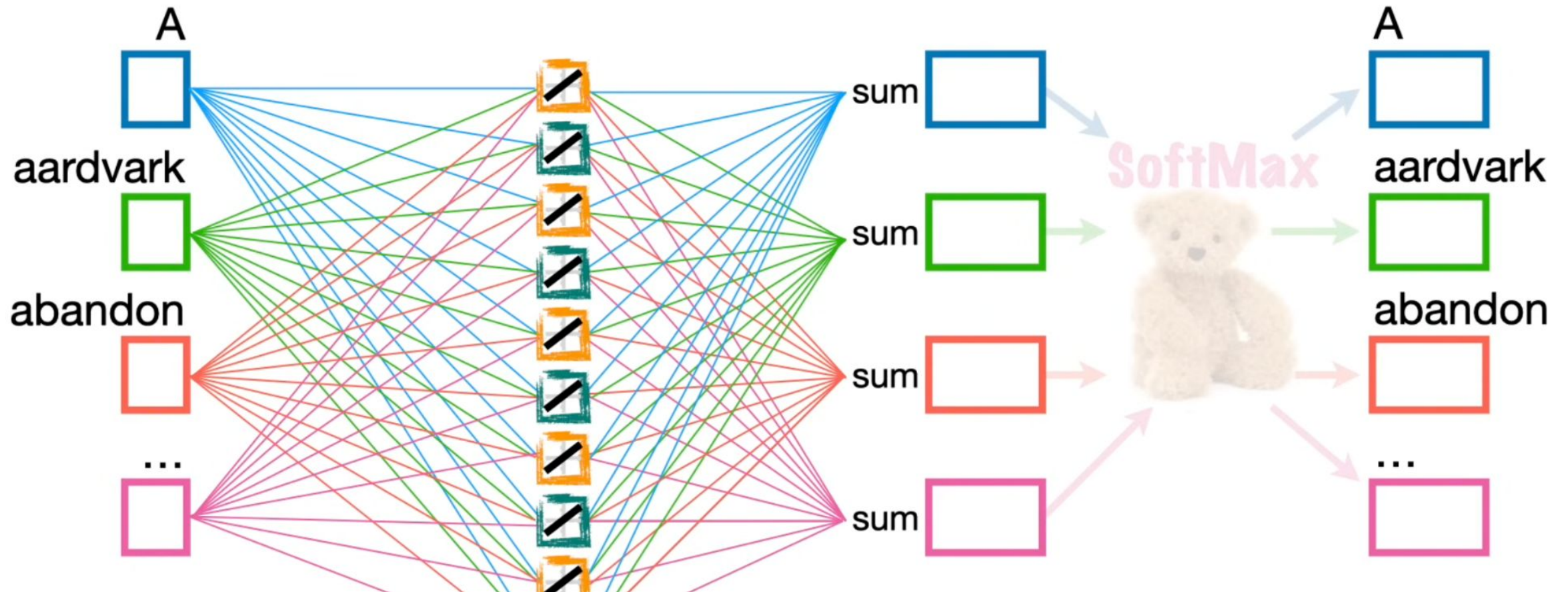
Feature Extraction

1. **Bag of Words (BoW):** Represents a document as a vector of word frequencies without considering word order.
2. **TF-IDF:** Assigns weight to a word based on how often it appears in the document (TF) and how rarely it appears across the entire corpus (IDF), so that common words receive low weights and distinct/unique words receive high weights.
3. **Word Embeddings (ex: Word2Vec, GloVe, etc.):** Maps each word to a dense, low-dimensional numerical vector that captures the semantic meaning and similarities between words.

Word Embeddings

- Traditional methods (Bag of Words, TF-IDF)
 - Create sparse, high-dimensional vectors
 - Treat word as isolated units
 - They lose the context and semantic meaning.
- **Word embeddings (Word2Vec, GloVe, Fast Text)**
 - Create dense, lower-dimensional vectors
 - These vectors capture the context and semantic relationships between words
 - The exact vectors are learned by the model (for example: Word2Vec)

Word Embedding



Word2Vec

2 Strategies:

1. Continuous Bag of Words: Predicts the word based on its surrounding context words.
2. Skip-gram: Does the reverse, predicting surrounding contexts from a single input words.

Optimization:

1. Negative Sampling: Randomly selecting word that don't want to predict.

```
nlp = gensim_api.load("word2vec-google-news-300")
```

```
## start the matrix (length of vocabulary x vector size) with all 0s
embeddings = np.zeros((len(dic_vocabulary)+1, 300))

for word,idx in dic_vocabulary.items():
    ## update the row with vector
    try:
        embeddings[idx] = nlp[word]
    ## if word not in model then skip and the row stays all 0s
    except:
        pass
```

<https://towardsdatascience.com/>

```
## input
x_in = layers.Input(shape=(15,))

## embedding
x = layers.Embedding(input_dim=embeddings.shape[0],
                     output_dim=embeddings.shape[1],
                     weights=[embeddings],
                     input_length=15, trainable=False)(x_in)
```

```
## 2 layers of bidirectional lstm
x = layers.Bidirectional(layers.LSTM(units=15, dropout=0.2,
                                     return_sequences=True))(x)
x = layers.Bidirectional(layers.LSTM(units=15, dropout=0.2))(x)

## final dense layers
x = layers.Dense(64, activation='relu')(x)
y_out = layers.Dense(3, activation='softmax')(x)

## compile
model = models.Model(x_in, y_out)
model.compile(loss='sparse_categorical_crossentropy',
              optimizer='adam', metrics=['accuracy'])

model.summary()
```

Dimensionality Reduction (ex: SVD)

```
from sklearn.decomposition import TruncatedSVD

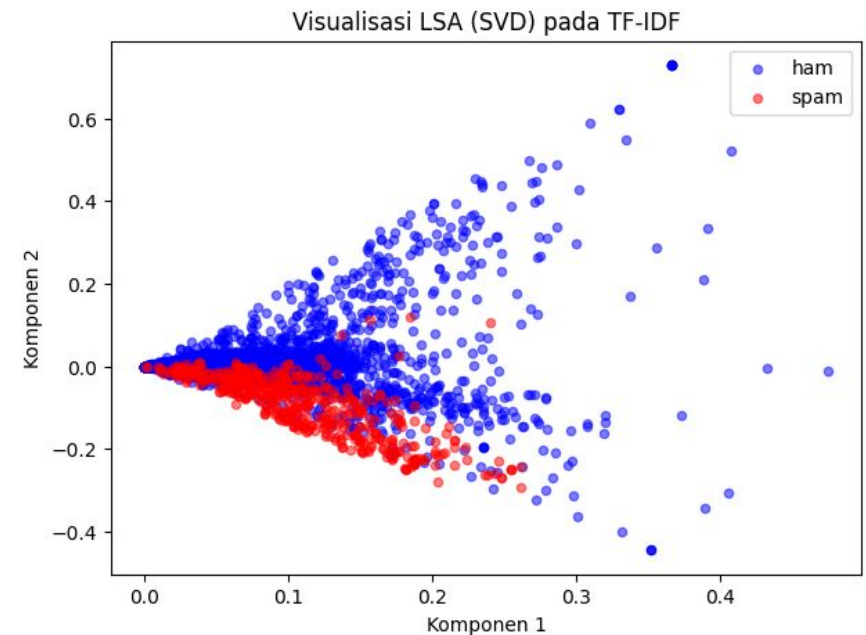
# Reduksi TF-IDF ke 2 dimensi
svd = TruncatedSVD(n_components=2, random_state=RANDOM_STATE)
X_svd_train = svd.fit_transform(X_tfidf_train)
X_svd_test = svd.transform(X_tfidf_test)

print("SVD shapes (train, test):", X_svd_train.shape, X_svd_test.shape)
```

TF-IDF shapes (train, test): (4126, 6169) (1032, 6169)



SVD shapes (train, test): (4126, 2) (1032, 2)



Code examples in python

- <https://www.kaggle.com/lystdo/lstm-with-word2vec-embeddings>
- <https://www.kaggle.com/hamishdickson/bidirectional-lstm-in-keras-with-glove-embeddings>

Lab Work 2

Write a sentiment text classification program code with the following specifications:

1. Use the dataset and template provided: bit.ly/LabWork2KitIF5153
2. Compare several word embeddings: **GloVe, Word2Vec, and FastText**
3. Compare them with SVD
4. Compare several non-transfer Deep Learning Algorithms: **CNN (1), RNN (1), and LSTM (1)**

Submit to Kaggle: <https://www.kaggle.com/t/02c30e5b245d434a8ba83f5b242401f9>

*** You must have at least one submission before the class ends (12 PM today).**

*** Change the Team Name on the kaggle competition to your Student ID (NIM).**

Create an analysis report containing:

- An explanation of the reasoning behind the chosen preprocessing techniques.
- A comparison of the results from the word embedding techniques and their impact on model performance.
- Insights gained.
- Ensure that the reports are complete and concise.

Lab Work 2

Deliverables:

1. Report named “02-[NIM]-report.pdf”
2. Notebook named “02-[NIM]-notebook.ipynb”
3. Zip the Report+Notebook into a single file named “02-[NIM].zip”

*If you do the lab work on Google Colab, please include the link in the report too. (ensure that the access is “*anyone with the link*” set as “*viewer*”)

*You may use english/indonesian for the report.

Deadline: Sunday, 13 September 2025, 23:59

References

- Word2Vec:
<https://medium.com/@manansuri/a-dummys-guide-to-word2vec-456444f3c673>
- Glove:
<https://www.geeksforgeeks.org/nlp/Glove-Word-Embedding-in-NLP/>
- FastText:
<https://www.geeksforgeeks.org/nlp/word-embeddings-using-fasttext/>